

Lab 4: System Calls for Basic Process Management

1. Commands:

```
ls
mkdir process
cd process
nano basic.c
```

Write in nano editor:

```
#include <stdio.h>
#include <time.h>
#include <unistd.h>

int main(){
    float pi;
    pi=3.1415;
    printf("\n The value of pi is %.4f\n",pi);
    return 0;
}
```

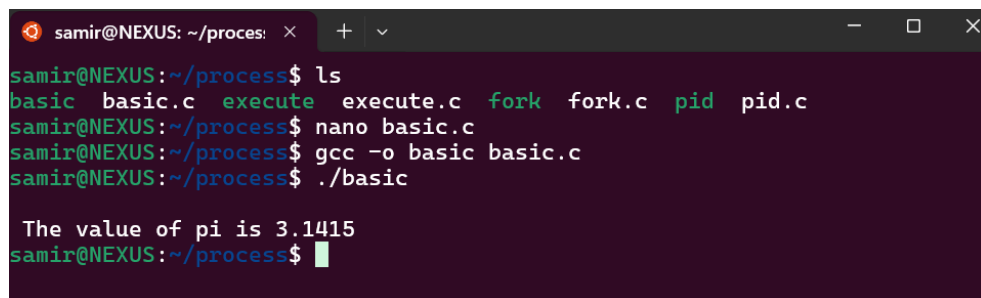
Press Ctrl+X to save.

Compile and run:

```
gcc -o basic basic.c
./basic
```

Interpretation: Prints the value of given constant.

Output:

A terminal window titled 'samir@NEXUS: ~/proces' with a dark background and light-colored text. The terminal shows the following commands and output: 'ls' lists files 'basic', 'basic.c', 'execute', 'execute.c', 'fork', 'fork.c', 'pid', and 'pid.c'; 'nano basic.c' opens the file; 'gcc -o basic basic.c' compiles the program; and './basic' runs it, outputting 'The value of pi is 3.1415'.

```
samir@NEXUS: ~/proces$ ls
basic  basic.c  execute  execute.c  fork  fork.c  pid  pid.c
samir@NEXUS: ~/proces$ nano basic.c
samir@NEXUS: ~/proces$ gcc -o basic basic.c
samir@NEXUS: ~/proces$ ./basic

The value of pi is 3.1415
samir@NEXUS: ~/proces$
```

2. Commands:

```
nano fork.c
```

Write in nano text editor:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    // make two process which run same program after this instruction
    if(fork()==0){
        printf("I am a child.\n");
    }
    else {
        printf("I am the parent.\n");
    }
    printf("Hello\n");
    return 0;
}
```

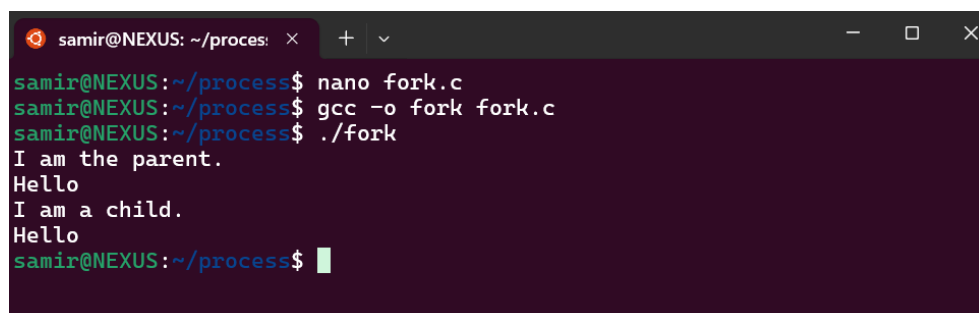
Press Ctrl+X to save.

Compile and run:

```
gcc -o fork fork.c
./fork
```

Interpretation: Create child process from the parent process.

Output:

A terminal window titled 'samir@NEXUS: ~/proces: x' with standard window controls. The terminal shows the following commands and output:

```
samir@NEXUS:~/process$ nano fork.c
samir@NEXUS:~/process$ gcc -o fork fork.c
samir@NEXUS:~/process$ ./fork
I am the parent.
Hello
I am a child.
Hello
samir@NEXUS:~/process$
```

3. Commands:

```
nano pid.c
```

Write in nano text editor:

```
#include <stdio.h>
#include <unistd.h> //contains fork prototype
```

```

int main()
{
    int pid,pid2;
    pid = fork();
    if (pid == 0)
    {
        printf("\nChild Process id : %d",getpid());
        printf("\nChild Process with parent id : %d \n",getppid());
    }
    pid2=fork();
    if(pid2==0){
        printf("\nChild Process id : %d",getpid());
        printf("\nChild Process with parent id : %d \n",getppid());
    }
    return 0;
}

```

Press Ctrl+X to save.

Compile and run:

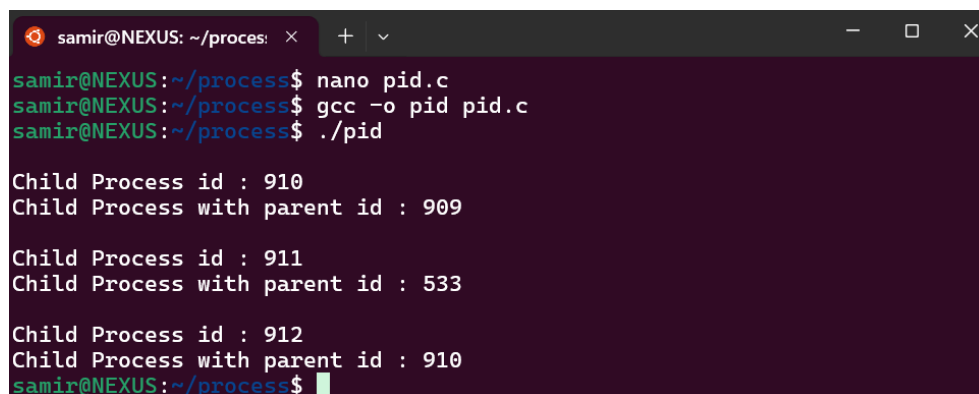
```

gcc -o pid pid.c
./pid

```

Interpretation: Use of fork to create child process from parent process.

Output:



```

samir@NEXUS: ~/proces: × + ▾
samir@NEXUS:~/process$ nano pid.c
samir@NEXUS:~/process$ gcc -o pid pid.c
samir@NEXUS:~/process$ ./pid

Child Process id : 910
Child Process with parent id : 909

Child Process id : 911
Child Process with parent id : 533

Child Process id : 912
Child Process with parent id : 910
samir@NEXUS:~/process$

```

4. Commands:

```

nano execute.c

```

Write in nano text editor:

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

```

```

int main() {
    char *temp[] = {NULL,"hello","world",NULL};
    temp[0]="basic";
    if(fork()==0){
        printf("I am a child process.\n");
        printf("My id is: %d\n",getpid());
        printf("My parent is: %d\n",getppid());
        printf("I am going to execute a c program\n");
        execve("basic",temp,NULL);
    }
    else{
        printf("The id of current process is %d\n",getpid());
    }
    return 0;
}

```

Press Ctrl+X to save.

Compile and run:

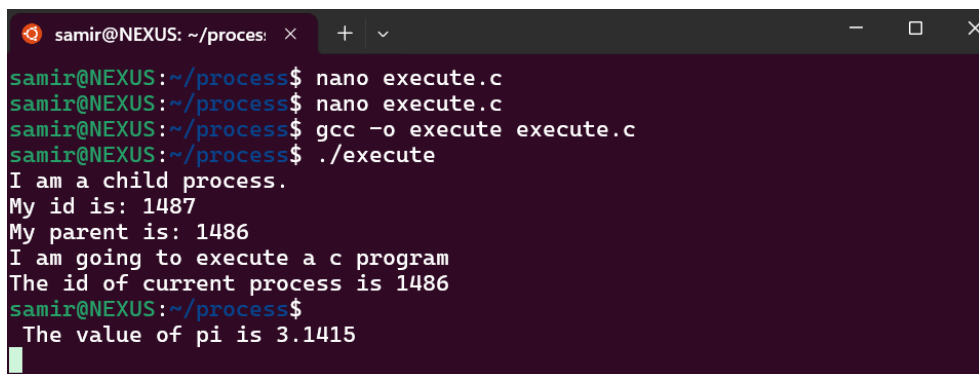
```

gcc -o execute execute.c
./execute

```

Interpretation: This program demonstrates the use of the `execve()` system call in Linux. When the program is executed, it creates a child process using `fork()`. The child process sets up an array of strings called "temp" which is used as an argument list for the `execve()` function.

Output:



```

samir@NEXUS: ~/proces: x + v
samir@NEXUS:~/process$ nano execute.c
samir@NEXUS:~/process$ nano execute.c
samir@NEXUS:~/process$ gcc -o execute execute.c
samir@NEXUS:~/process$ ./execute
I am a child process.
My id is: 1487
My parent is: 1486
I am going to execute a c program
The id of current process is 1486
samir@NEXUS:~/process$
The value of pi is 3.1415

```